INFORMATIONSBLATT

IB02 – Eingaben sicher verarbeiten

Dynamische Webseiten mit PHP · LS03 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie wandeln Formulareingaben in den passenden Datentyp um, entfernen überflüssige Leerzeichen, formatieren Datumsangaben und geben Eingaben so aus, dass niemand fremdes HTML in Ihre Seite schmuggeln kann.

1 | Alles ist ein String

Was aus einem Formular kommt, ist **immer Text** – auch bei `type="number"`. Für Rechnungen wandelt man den Wert deshalb um (*Typumwandlung*, englisch *cast*):

Typumwandlung beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Alles ist Text</title></head>
<body>

<pre>
<?php
  var_dump($_POST['personen']);
  var_dump((int) $_POST['personen']);

  $betrag = str_replace(",", ".", $_POST['betrag']);
  var_dump((float) $betrag);
?>
</pre>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS

localhost/beispiele/beispiel.php

```
string(1) "4"
int(4)
float(2.5)
```

Abbildung 1: Aus dem String "4" wird die Zahl 4, aus "2,5" die Kommazahl 2.5.

Umwandlung	Ergebnis	Hinweis
`(int) "4"`	4	
`(int) "4 Personen"`	4	liest Ziffern bis zum ersten anderen Zeichen
`(int) "abc"`	0	kein Fehler, aber 0 – später prüfen!
`(float) "2.5"`	2.5	
`(float) "2,5"`	2	das Komma beendet die Zahl – vorher mit `str_replace(",", ".", …)` ersetzen

2 | Leerzeichen abschneiden mit trim()

Beim Kopieren und Einfügen landen oft unsichtbare Leerzeichen im Feld: `" anna@beispiel.de "`. `trim()` entfernt Leerzeichen, Tabulatoren und Zeilenumbrüche am Anfang und am Ende:

```
trim()

$mail = trim($_POST['mail']);     // " anna@beispiel.de " -> "anna@beispiel.de"
```

3 | Eingaben sicher ausgeben mit htmlspecialchars()

Was ein Besucher eintippt, gibt PHP eins zu eins in die Seite aus. Enthält die Eingabe HTML-Tags, führt der Browser sie aus. Hier trägt jemand statt eines Namens eine HTML-Überschrift ein:

```
Unsicher: Eingabe direkt ausgeben                                beispiel.php

<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Unsicher</title></head>
<body>

<?php
  $name = $_POST['name'];
  echo "<p>Vielen Dank, $name!</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS



Abbildung 2: Ohne Schutz wird die Eingabe als HTML ausgeführt.

Sicher: Eingabe entschärfen beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Sicher</title></head>
<body>

<?php
  $name = htmlspecialchars($_POST['name']);
  echo "<p>Vielen Dank, $name!</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

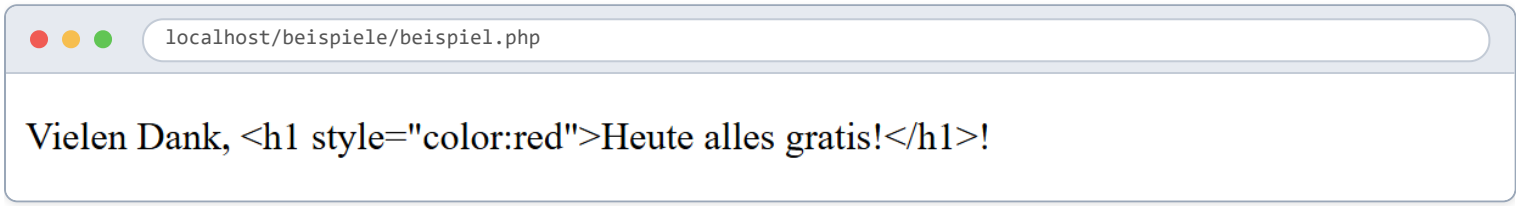


Abbildung 3: Mit htmlspecialchars() erscheint die Eingabe als harmloser Text.

`htmlspecialchars()` ersetzt die Zeichen, die in HTML eine Bedeutung haben, durch ihre Umschreibung. Der Browser zeigt sie dann an, statt sie auszuführen:

Zeichen	wird zu
`<`	`<`
`>`	`>`
`"`	`"`
`&`	`&`

⚠ CROSS-SITE-SCRIPTING (XSS)

Eine rote Überschrift wirkt harmlos. Statt HTML kann ein Angreifer aber auch **JavaScript** einschleusen, das im Browser anderer Besucher läuft – etwa um Anmeldedaten abzugreifen. Dieser Angriff heißt *Cross-Site-Scripting* und gehört seit Jahren zu den häufigsten Sicherheitslücken im Web.

Regel: Jede Benutzereingabe, die in einer Seite ausgegeben wird, läuft durch `htmlspecialchars()`.

4 | Datum umformatieren

Ein Feld `<input type="date">` liefert das Datum immer im internationalen Format `JJJJ-MM-TT`. Mit zwei Funktionen wird daraus das deutsche Format:

2027-07-24 wird 24.07.2027 beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Datum</title></head>
<body>

<?php
  $datum = "2027-07-24";                    // so liefert <input type="date"> das Datum
  $datum_de = date("d.m.Y", strtotime($datum));
  echo "<p>Ihr Termin: $datum_de</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

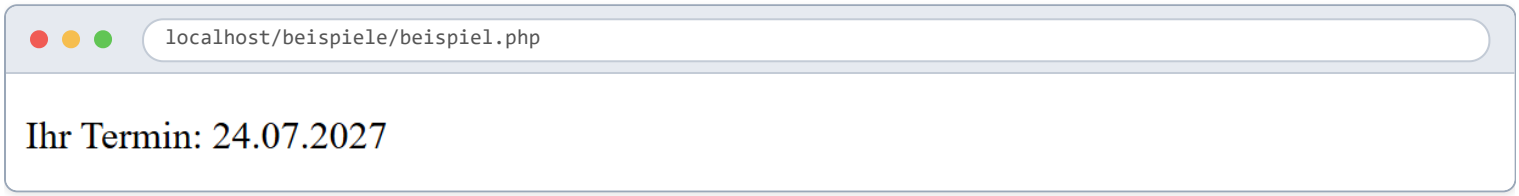


Abbildung 4: strtotime() versteht das Datum, date() gibt es neu formatiert aus.

Funktion	Aufgabe
`strtotime("2027-07-24")`	wandelt einen Datumstext in einen *Zeitstempel* um (Sekunden seit 1.1.1970)
`date("d.m.Y", $zeitstempel)`	formatiert diesen Zeitpunkt statt des aktuellen – das bekannte Formatmuster aus LS01

✅ DER DREISCHRITT FÜR JEDE EINGABE

1. **Übernehmen und bereinigen:** `trim()`, Typumwandlung `(int)` / `(float)`
2. **Prüfen:** Ist der Wert vorhanden und sinnvoll? – ab LS04
3. **Sicher ausgeben:** `htmlspecialchars()`